

Static and Dynamic Hand Gesture Recognition in Depth Data Using Dynamic Time Warping

Guillaume Plouffe and Ana-Maria Cretu, *Member, IEEE*

Abstract—This paper discusses the development of a natural gesture user interface that tracks and recognizes in real time hand gestures based on depth data collected by a Kinect sensor. The interest space corresponding to the hands is first segmented based on the assumption that the hand of the user is the closest object in the scene to the camera. A novel algorithm is proposed to improve the scanning time in order to identify the first pixel on the hand contour within this space. Starting from this pixel, a directional search algorithm allows for the identification of the entire hand contour. The k -curvature algorithm is then employed to locate the fingertips over the contour, and dynamic time warping is used to select gesture candidates and also to recognize gestures by comparing an observed gesture with a series of prerecorded reference gestures. The comparison of results with state-of-the-art approaches shows that the proposed system outperforms most of the solutions for the static recognition of sign digits and is similar in terms of performance for the static and dynamic recognition of popular signs and for the sign language alphabet. The solution simultaneously deals with static and dynamic gestures as well as with multiple hands within the interest space. An average recognition rate of 92.4% is achieved over 55 static and dynamic gestures. Two possible applications of this work are discussed and evaluated: one for interpretation of sign digits and gestures for a friendlier human-machine interaction and the other one for the natural control of a software interface.

Index Terms—Dynamic gestures, hand gesture recognition, human-computer interaction, Red Green Blue - Depth (RGB-D) sensor, static gestures.

I. INTRODUCTION

RECENT years have witnessed an increased research interest in the development of natural intuitive user interfaces. Such interfaces have to remain invisible to the users, allowing them to unobtrusively interact with an application, without the need for specialized and costly equipment. They have to support a natural interaction and be adaptable to the user without imposing elaborate calibration procedures. At the same time, they have to fulfill their role in real time, with accuracy, and provide robustness against background clutter. These various requirements and their inherent complexity still provide significant challenges for researchers.

As hand gestures constitute a powerful interhuman communication modality, they can be considered as well an intuitive

and convenient mean for the communication between human and machines. This justifies the interest of the research community in the development and advancement of hand gesture technologies. One of the most important abilities of an efficient natural user interface is therefore its ability to recognize in real time hand gestures.

The principal components of a hand recognition system are data acquisition, hand localization (e.g., segmentation and tracking), hand feature identification, and gesture recognition based on identified features. The classic solution for data acquisition is color cameras that have already been successfully employed for gesture recognition tasks [1]–[3]. These solutions are, however, sensitive to clutter, lighting conditions, and skin color. Video capture has the extra challenge related to the speed of movement. In terms of 3-D motion capture at the level of the fingers, possible solutions include optical marker systems, accelerometers, magnetic trackers, and data gloves. These require extensive calibration, limit the natural movement of the fingers, and are generally very expensive. The vision-based markerless motion capture became possible largely due to the introduction on the market of the Kinect, a low-cost off-the-shelf sensor that gathers positional information about an individual's motion. Data captured by this sensor, in RGB-D (color and depth information) form, are often used as a source for hand gesture detection and recognition. While a good survey for the gesture recognition in general is available in [4] and one focused on depth images in [5], the following literature review briefly summarizes the solutions that make use of RGB-D data for the purpose of hand gesture recognition.

Different approaches can be employed for hand localization in acquired data, according to their nature. For depth data, the classical solution is depth thresholding, either empirical or automated. Empirical solutions choose the limits of the most probable search space by trial and error and concentrate the computation effort on the hand localization within it. Among the automated solutions are the detection of the closest point to the camera [6]–[10] based on the assumption that the hand is the closest object in the scene and the use of other reference elements in the scene, such as the head position [11]–[13] or the facial color information [3] to identify the most possible location of the hand. An exhaustive solution is to scan the entire visibility space of the Kinect using overlapping depth intervals until the hands are found [14]. This can be, however, a very lengthy process. In terms of color images, possible solutions for hand segmentation include skin color detection [1], [3], [12], [15]–[17] and Haar-like features [2].

Manuscript received May 3, 2015; revised August 27, 2015; accepted September 21, 2015. Date of publication November 20, 2015; date of current version January 4, 2016. This work was supported by the Natural Sciences and Engineering Research Council of Canada. The Associate Editor coordinating the review process was Dr. Shervin Shirmohammadi.

The authors are with the Department of Computer Science and Engineering, Université du Québec en Outaouais, Gatineau, QC J8X 3X7, Canada (e-mail: ana-maria.cretu@uqo.ca).

Color versions of one or more of the figures in this paper are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/TIM.2015.2498560

0018-9456 © 2015 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.

See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

Finally, certain approaches in the literature benefit from both depth and color information [7], [15]–[22]. Some systems impose the intervention of the user to calibrate the system prior to its use; for example, in [20], the location of the hand is recuperated from a wave gesture executed by the user. Others impose constraints, such as the fact that the user wears a colored glove [22] or a black belt on the wrist of the gesturing hand [7], [18], [23] in order to simplify the hand localization procedure and its segmentation. These solutions impede on the invisibility of the interface to the user. Finally, Ryan [14] favors the middle of the screen as an area of higher likelihood for hand detection, therefore avoiding the search along the image sides in order to improve the response time. Such a system constrains the user to adapt its behavior to the interface.

After the hand localization, the next step is the selection of appropriate features that allow for the recognition of gestures. Among the solutions encountered in the literature for depth data are hand contour, hand palm center [6], [14], finger properties (e.g., count and direction) [24], hand trajectory [11], [25], and gradient kernel descriptors [21], while for color data, popular choices are Haarlet features [20], Gabor features [26], [27], Scale Invariant Feature Transform (SIFT) [21], Speeded Up Robust Features (SURF) [15], Hu moment invariants [17], Histogram of Oriented Gradients (HOG) [28], [29] descriptors, and shape distance metrics, such as the finger–earth mover’s distance [7], [18], [19]. Otiniano-Rodríguez and Cámara-Chávez [21], Abid *et al.* [28], and Dardas and Georganas [30] combine features using a bag-of-visual-words approach prior to recognition. For the recognition of gestures from the selected features, possible techniques proposed in the literature are neural networks [11], [20], Support Vector Machines (SVMs) [15], [28], [30], nearest neighbor classifiers [8], [10], random forests [26], fuzzy approaches [1], and hidden Markov models [15], [27], [29]. Alternatively, a simple method that takes into consideration the position of middle and end points of a gesture trajectory with respect to the initial points is proposed in [17]. Most of the existing solutions for gesture recognition are designed for use either on static [8], [10], [20], [21], [24], [26], [31], [32] or on dynamic gestures [11], [28], [33], [34]. Overall, there are only very few solutions that simultaneously work on both static and dynamic gestures [12], [25], [27]. Moreover, most of the current solutions are able to handle a single hand [1], [7], [15], [18], [21], [26], [29], [31], [32] or two hands [14], [17], [24]. To the best of our knowledge, there are no solutions that can track and recognize gestures from multiple hands, as the one proposed in this paper. The results, in most cases in form of average recognition rates, are presented on data sets of selected static and/or dynamic gestures varying from 6 to 42 gestures: 6 in [32], 10 in [8], 12 in [10], 14 in [17] and [24], 24 (or 26) [gestures of the American sign language (ASL) alphabet; two gestures in this alphabet are dynamic and some authors work on static gestures only] in [9] and [21], and 42 gestures in [27]. Most results are reported for a single hand [7], [9], [27], [34] and a few for two hands [17], [24].

The objective of this paper is to develop an improved, low-complexity, and real-time solution for the recognition of both static and dynamic gestures executed by one or

multiple hands from depth data returned by a Kinect sensor. The gain in performance and speed comes from multiple improvements in the various stages of the solution, including a faster algorithm to search the first pixel of the hand contour; the recognition of gestures independent of their position in the field of view of the Kinect; the use and adaptation of dynamic time warping (DTW) not only as a validation tool [14] but also as a selector for candidate gestures; the normalization of disparity values to improve the recognition rate in the case of multiple hand detection; the use of constraints on the contour to ensure it is closed; and the use of contour information for recognition only in cases when no fingertips are identified, in order to improve the computation time. Moreover, the proposed approach offers an interface that allows the user to train the system with his/her desired gestures and rapidly test its performance. According to a recent review on gesture recognition [5], this is one missing aspect of current solutions: the capacity to learn a gesture set from the user. The interface is invisible to the user in the sense that it does not require any constraints or calibration procedure. Due to the fact that our system is based only on depth information, cluttered backgrounds, lighting conditions, clothing, or skin color have little impact on the reported performance.

Gesture recognition has multiple applications in the context of sign language interpretation [21], [26], [27], smart home interactive control [33], control of a software interface [18], [19], [35] control of virtual environments [15], human–robot interaction [20], [36], and robot hand control [22]. Among the possible applications of this work, two are evaluated in the context of this paper: one for the real-time interpretation of sign language digits and gestures and the other one in the context of natural control of a software application.

II. PROPOSED SOLUTION FOR HAND DETECTION AND GESTURE RECOGNITION IN DEPTH DATA

The proposed framework for hand detection and gesture recognition is illustrated in Fig. 1. Raw data coming from the Kinect are used to recuperate depth information on all the pixels of an image. A novel faster algorithm is then proposed to identify each point of a closed contour identified within a given depth interval.

Starting from the recuperated contour points, the center of the palm is identified as the center of the largest circle circumscribed in the contour. The fingertips are localized by employing the k -curvature algorithm, which is based on the change in the slope angle of the tangent line at selected points over the contour. During the execution, the hand information (contour, palm center, and fingertips) is stored in a fixed-size First In First Out (FIFO) list. This list contains data on reference gestures that can be recorded by a user, according to the specific needs of the application. The same list is also used by the DTW algorithm in order to recognize previously recorded reference gestures.

A. Contour Detection and Tracking

From the 16-bit raw data collected by the Windows Software Development Kit (SDK) 1.8 for Kinect,

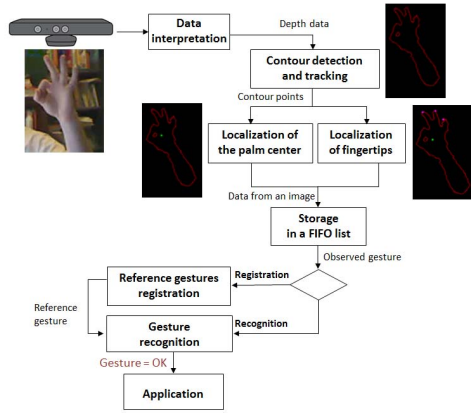


Fig. 1. Proposed framework for hand detection and gesture recognition.

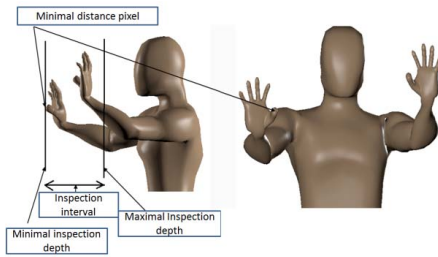


Fig. 2. Inspection interval with respect to minimum distance pixel.

the last 13 bits represent the depth information. A 3-bit shift operation is performed to retain only this information. Each time a new frame becomes available from the Kinect, the depth information is recuperated and used to detect one or multiple hand contours.

1) *Detection of the Interest Space:* In order to guide the search for the hand contour, the proposed solution starts by identifying the minimal distance (closest) pixel to the Kinect, as illustrated in Fig. 2, based on the assumption that the hand is the closest object to the camera in the scene. The depth of this pixel is considered to represent the minimal depth for inspection, and a constant (its value corresponding to the interval of inspection, i.e., an empirically determined value of 20 cm in this paper) is added in order to calculate the maximum depth within which a search is performed for the hands. The search for the hand(s) occurs only within this limited space.

This approach is simple and does not impose significant limitations for the user, as we naturally tend to keep the hands in front of the body when we gesticulate. In order not to have to be constrained by the closest pixel, a possible solution is to use other elements in the Kinect image as a reference. For example, the skeleton tracking feature of Kinect could be employed to identify the hand position as a reference element. However, due to the fact that this joint is among the least stable joints to track using the Kinect SDK [37], this solution was not retained in this work.

2) *Search of the First Pixel of a Contour:* A search is performed within the inspection interval to identify the first pixel that has at least one neighboring pixel that is not

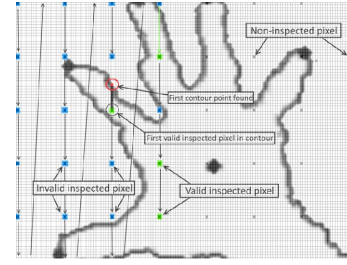


Fig. 3. Proposed algorithm for the detection of the first point of the contour inspecting each block of 20×20 pixels.

included in the inspection interval. To optimize the search, [14] proposes to inspect every other 5 pixels (e.g., blocks of 5×5) and does not take into consideration the pixels around the edges of the image (20% of pixel lines). In this paper, the search algorithm is improved as follows: a pixel is considered valid for inspection if it is present at a depth inside the inspection interval illustrated in Fig. 2. Any other pixel is ignored in the analysis of the frame. The search takes place in blocks of 20×20 pixels. If a pixel is valid and if it does not possess any neighboring pixel that is valid, the search continues pixel by pixel toward an invalid neighboring pixel until a contour pixel is found, as illustrated in Fig. 3. We verify if this pixel is part of an already found contour and, if not, a new potential hand is found.

If a pixel is valid and none of the neighboring pixels are invalid, the search continues with the next block. This algorithm results in a faster search. Because the Kinect is used with a resolution of 640×480 pixels, each image contains 307200 pixels. In the worst case, the proposed algorithm covers maximum 768 pixels ($307200/400$) to confirm that an image does not contain any hand. An improvement of 99.75% [or $100 - (768/307200) \times 100$] is therefore achieved for an image of 640×480 with respect to a full search. The proposed solution improves the search time by 15.25% with respect to the solution proposed in [14], and unlike the latter, it does not restrict the search to the central part of the image.

3) *Contour Point Detection:* Once the initial contour pixel is found, a directional search is performed to identify the full contour of the hand, as in [14]. In the context of this paper, the considered directions are upper left, upper right, bottom left, and bottom right. A search direction is favored if it has been used for the previous pixel as well. The algorithm also includes a solution to backtrack if an unknown valid configuration is encountered. To further improve this approach, in this paper, a constraint is added to validate only closed contours, as illustrated in the pseudocode of Algorithm 1.

The detected contour using this procedure for the OK sign in Fig. 4(a) is shown in Fig. 4(b). In order to identify more than one hand, once the closed contour points of the first hand are discovered, the algorithm continues the search for the first pixel of the next contour and repeats the process until it reaches the end of the image. The pseudocode above is therefore repeated in a *for* loop for the detection of multiple hands.

Algorithm 1 Contour Point Detection

Input:

The first point, P_1 , of a list of contour points belonging to a potential hand contour, C

Output:

List of contour points P_i belonging to an identified contour C For each potential hand contour C From the first point P_1 found, trace in order each point of contour and stop when the $C.length > 700$ or when the next point is already present in the contour list.

// if the last point is near the first point

if $((C.length > 10 \text{ and } (|LastPoint.x - P_1.x| \leq 15 \text{ pixels and } |LastPoint.y - P_1.y| \leq 15 \text{ pixels and } |LastPoint.z - P_1.z| \leq 15 \text{ pixels}))$ thenthe Contour C is accepted

Display the contour points in red

else

the Contour C is rejected

end

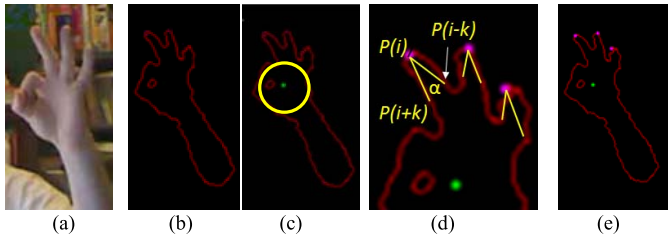


Fig. 4. (a) Initial OK gesture. (b) Detected contour. (c) Largest circle contained in the palm (yellow circle) and identified palm center (green dot). (d) Angle calculation for fingertip direction computation. (e) Identified fingertips (magenta dots).

B. Hand Palm Center Localization

Similar to the approach in [6], the hand palm center is calculated using the contour pixels as well as the interior pixels of the hand. To identify the bounding box of the hand, the minimum and maximum points on the X and Y axes are computed. The minimal distance (d_{min}) of interior pixels is then calculated for each block of 5×5 pixels with respect to each contour pixel. One contour pixel in five was experimentally proved to be sufficient for the final estimation of the hand center. The interior pixel that has the maximal value for d_{min} is considered to be the center of the palm.

In simple terms, the center of the palm is the center of the largest circle that can be circumscribed in the palm, as illustrated in Fig. 4(c) for the case of the OK sign. The proposed approach is summarized in the pseudocode of Algorithm 2.

C. Localization of Fingertips

The k -curvature algorithm is employed to identify the fingertips over the hand contour. The choice of this algorithm is justified by the fact that recent research showed that when compared with other solutions for static hand recognition, it provided the best results in terms of overall success rate, supported the highest range of hand rotations within which it is

Algorithm 2 Palm Center Localization

Input:

 C = List of contour point from the hand (obtained using Algorithm 1) I = List of points inside the contour

Output:

Index of palm center coordinates in list I 1. For each element i of I :For each element c of C :Calculate the Euclidian distance d_i between i and c If $d_i > d_{i \min}$ then $d_{i \min} = d_i$ If $d_{i \min} < d_{i \max}$, then

Return to step 1

end

Return the minimum Euclidean distance value obtained for i , $d_{i \min}$

end

2. Return the center of palm = element i of I with the maximum Euclidean distance value obtained, $d_{i \max} = \max(d_{i \min})$.

Display a green circle at the coordinates of the palm center.

capable of performing reliably, and had the lowest complexity in terms of specific steps [31]. In simple terms, and as illustrated in Fig. 4(d), the algorithm uses two equal-length measuring tapes, V_A and V_B , of 20 pixels each (e.g., step size, $k = 20$ pixels) and measures at each point the angle α between them [e.g., between $P(i-k) \rightarrow P(i)$ and $P(i) \rightarrow P(i+k)$] over the contour of the hand. If the angle has a value between 25° and 55° , a fingertip is considered identified at that point. These angle limiting values are experimentally identified by trial and error. The pseudocode of the algorithm is summarized in Algorithm 3, and the detected fingertips for the OK sign are shown in magenta dots in Fig. 4(d) and (e).

While sometimes this algorithm can lead to false fingertip detection, particularly when the fingers of the user are thin and/or when they are very close to each other, the situation was not encountered frequently enough to be addressed. A possible solution to this problem would be to use an extra constraint on the distance between the center of the palm and the fingertips as well as between the fingertips. In addition, a solution could be added for the prediction of a missing finger by computing the mean of movements in previous consecutive images, as in [14]. All these would nevertheless have a negative impact on the hand tracking time, while not necessarily bringing significant detection improvements.

D. Gesture Recognition Using Dynamic Time Warping

While static recognition can be accomplished by standard pattern recognition techniques (e.g., template matching), dynamic gesture recognition requires the incorporation of a time component in the process. Among the most often employed techniques are time-compressing templates, DTW,

Algorithm 3 Fingertip Localization

Input:

$\mathbf{C}$ = List of hand contour points
 P_1 = first point of the contour $\mathbf{C}$
 c_A = Start point of vector $\mathbf{V}_A$
 c_B = End point of vector $\mathbf{V}_B$

Output:

Series of points P_f representing the detected fingertips $k = 20$; $c_A = P_1 - k$; $c_B = P_1 + k$ For each element c of $\mathbf{C}$:if $c < k$ then $c_A = \mathbf{C}.\text{length} - k + c$

else

 $c_A = c - k$

end

if $c > \mathbf{C}.\text{length}$ then $c_B = k - \mathbf{C}.\text{length} + c$

else

 $c_B = c + k$

end

create vector $\mathbf{V}_A(\mathbf{C}[c_A], \mathbf{C}[c])$ create vector $\mathbf{V}_B(\mathbf{C}[c], \mathbf{C}[c_B])$ calculate $\theta(c) = \arccos \left| \frac{\mathbf{V}_A \cdot \mathbf{V}_B}{\mathbf{V}_A.\text{length} * \mathbf{V}_B.\text{length}} \right|$ (in radians)transform $\theta(c)$ in degreesIf $\theta(c) \leq 55$ and $\theta(c) \geq 25$

// fingertip detected

display a magenta dot of coordinates c representing the fingertip

end

hidden Markov models, and time-delayed neural networks [4]. In the context of this work, the DTW algorithm is used to compare the similarity between a reference and a captured gesture, whether the gesture is static or dynamic. The proposed approach allows the user to record a sequence of reference gestures that are saved in an FIFO list. In order to ensure real-time behavior of the system, this list is limited to 40 images. When the recording is finished, these images are saved in an xml file. Once a sequence of new images representing a gesture being executed is made available by the Kinect, the recognition module is activated to recognize it based on the features identified in Sections II-B and II-C.

The gesture recognition module executes two steps: 1) the search of candidate gestures based on the similarity between the last image of the observed gesture and each reference gesture and 2) the validation of the similarity between the observed gesture and each identified candidate reference gesture using the DTW matrix. For the first step, [14] compares the last image of an observed gesture with the last image of each reference gesture in the list of reference gestures by calculating the Euclidian distance. If the reference gesture associated with the lowest distance is within a threshold, the gesture is considered to be a possible candidate. While simple and fast, this solution fails to distinguish between two dynamic gestures whose last image is identical, because it compares only the latter in order to recognize a gesture. To cope with

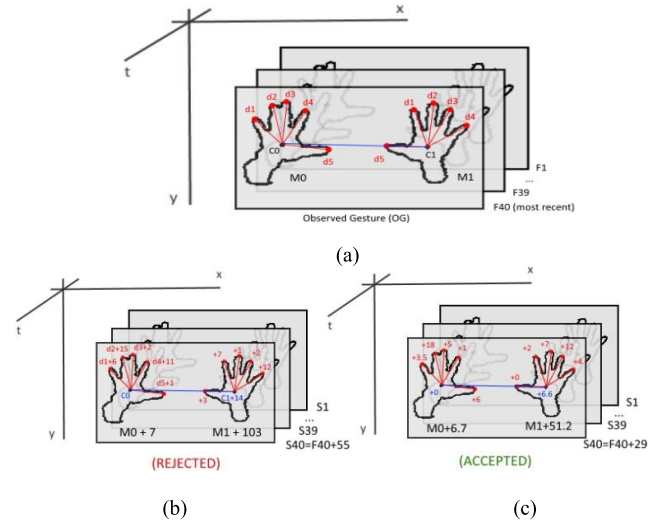


Fig. 5. Gesture disparity calculation between the (a) observed and (b) and (c) stored gestures for the selection of gesture candidates.

this problem, we propose the use of DTW algorithm [38], [39] not only for validation [in step 2)], but also as a mean to identify the best candidate among a list of possible candidates [in step 1)]. The impact of this choice is further illustrated in Section III.

The DTW algorithm calculates the disparity (inverse of similarity) between two data series acquired at different times. A matrix containing the Euclidian distances at aligned points over the two sequences is used to calculate the minimal cost (shortest path) between the two series. The choice of direction for the shortest path is associated with certain rules. In particular, the movement is restricted to horizontal, vertical, and diagonal directions. A weight is associated with each of these directions, and the shortest path has to be inferior to a threshold in order for the two sequences to be considered similar. In the context of this work, the two series represent the sequences of hand positions in an observed and a reference gesture, respectively. To allow for the recognition of gestures independent of their position on the screen, in step 1), we use the position of fingertips with respect to the center of the palm(s) and the relative position of the palm(s) center with respect to the palm center of the first discovered contour, as illustrated in Fig. 5(a). The hand disparity value is calculated as a sum of Euclidean distances between each fingertip and the palm center. For example, in order to select a candidate gesture, we compute the disparity between the 40th frame of an observed gesture [Fig. 5(a)] and of a stored gesture [Fig. 5(b)], using the following calculation:

$$\Delta SO_{40} = \frac{\sum_{j=1}^{M-1} \frac{\sum_{i=1}^{N-1} \Delta d_i}{N} \times w_f + \Delta C_j \times w_p}{M} \quad (1)$$

where M is the number of hands detected in the image, N is the number of fingers detected in the image, Δd_i is the difference in the distance from the palm center to the detected fingertip i in the observed and stored gesture frames, ΔC_j is the difference between the palm centers j in the stored and observed gesture frames, and w_f and w_p are the

TABLE I
PARAMETERS FOR THE DTW ALGORITHM

Parameter	Value
The maximum Euclidian distance, in pixels, accepted for a candidate gesture	30
The number of vertical or horizontal movements allowed in the shortest path (HorizontalMovementThreshold, VerticalMovementThreshold)	10
Maximum cost accepted for a path in the DTW matrix per image (PathCostThreshold)	30
Weight used for the difference between the relative palm position in the last image of the reference gesture and the one of the observed during candidate search	7
Weight used for the difference of palm position relative to the last image between each image of the candidate reference gesture and observed gesture at the construction of the DTW matrix	7
Weight used for the difference between the finger position with respect to the palm center in the last image of the reference gesture and the one of the observed during candidate search	1
Weight used for the difference of finger position with respect to the palm center between each image of the reference gesture and observed gesture at the construction of the DTW matrix	1
Weight used for direction choices (horizontal, vertical, diagonal) during the search of the shortest path	(0.005, 0.005, 0.003)

weights for the finger and palm center disparities, respectively (the values are listed in Table I). For frames with only one hand, the palm center disparity is always 0 since the position is relative to itself. For the situation illustrated in Fig. 5(b), (1) becomes

$$\Delta SO_{40} = \frac{\left(\left(\frac{(6+15+2+11+1)}{5} \times 1 \right) + 0 \times 7 \right) + \left(\left(\frac{(7+1+2+12+3)}{5} \times 1 \right) + 14 \times 7 \right)}{2} = 55.$$

For a limited number of gestures in which the fingertips are not visible, the contour points (each tenth point) are used as a basis to calculate the distance. Since this distance gets higher for gestures involving multiple hands and fingers, a normalization operation is applied by the number of fingers, N , detected in an image in order not to bias the recognition. For the same reason, the disparity value for an entire image is obtained by dividing the total hand disparity over all detected hands by the number of hands, M . If this value is lower than a threshold (e.g., the maximum Euclidian distance accepted for a candidate gesture in Table I), the gestures are considered similar [Fig. 5(c): $29 < 30$] or else they are rejected as possible candidates [Fig. 5(b): $55 > 30$].

In step 2), we use the relative position of the palm(s) center with respect to the corresponding palm center of the last recorded image, as illustrated in Fig. 6, to validate the similarity between the observed gesture and each identified candidate reference gesture. The gesture disparity is calculated as the sum of disparities for all images divided by the number of images. To validate the similarity, ΔSO , between a candidate and the observed gesture, we use the following formula based on the set of cells, P , illustrated in Fig. 6(b),

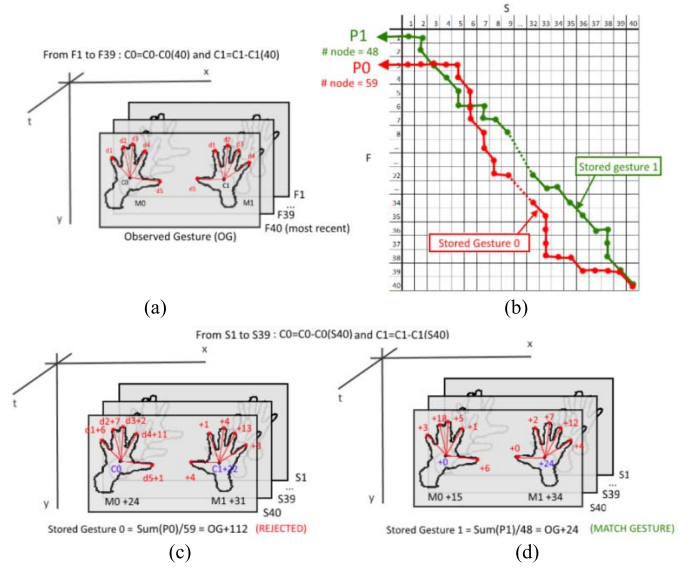


Fig. 6. Gesture disparity calculation for the validation of the recognized gesture between the (a) observed gesture and (c) and (d) stored gestures using the (b) DTW matrix.

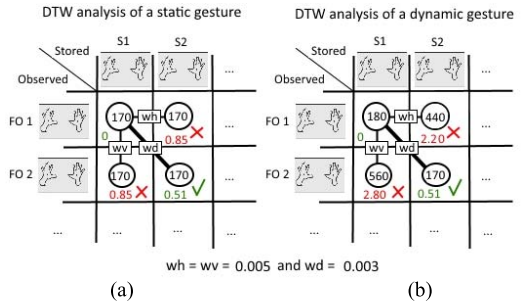


Fig. 7. DTW matrix for (a) static and (b) dynamic gestures.

representing the minimum cost path of the DTW matrix:

$$\Delta SO = \frac{\sum_{e=1}^{\#P} \Delta SO_e}{\#P} \quad (2)$$

where $e = \{(x, y) | x \in S, y \in F, (x, y) \in P\}$, $\#P$ represents the cardinality of the set P , S is the set of stored gesture frames, and F is the set of observed gestures frames, each set containing 40 frames. To calculate ΔSO_e in (2), we use the same formula for the selection of candidate (1), but with $\Delta d_{ije} = ||d_{ijx} - C_{jx}| - |d_{ijy} - C_{jy}||$ and $\Delta C_{je} = |||C_{jx} - C_{j=1,x}| - |C_{j,x=40} - C_{j=1,x=40}|| - ||C_{jy} - C_{j=1,y}| - |C_{j,y=40} - C_{j=1,y=40}||$, where i and j refer, respectively, to the finger and hand identification number, $x \in S$ and $y \in F$.

Table I summarizes the empirical choice of parameters used during experimentation for the DTW.

The value for the maximum Euclidian distance, in pixels, accepted for a candidate gesture is selected as 30. It represents a good compromise between the number of candidate gestures to be processed and the processing time, as in this case, the 40 last images in the list are compared to make a decision. The maximum cost accepted for a path in the DTW matrix per image is empirically set to 30. The weights associated with the directions of movement are experimentally set to 0.005 for vertical and horizontal directions and 0.003 for the diagonal direction, in order to favor the latter. When we perform

Algorithm 4 Gesture Recognition Using DTW

Input:

g_{obs} = Last observed gestures (40 images)**g_{cand}** = List of candidate gestures (40 images)

Output:

g_{recogn} = Name of recognized gesture (or none:
gesture not recognized)For each gesture in **g_{cand}**:// Get the 2D DTW matrix containing the disparity between
each

// image of both gestures.

dtwMatrix = FindDtwMatrix (**g_{obs}**, gesture)// Use a Greedy algorithm to find the minimum path cost
in disparities// in the DTW matrix; return $-\infty$ if the cost between two
images is ∞ // or the horizontalMovementCounter > HorizontalMove-
mentThreshold// or the verticalMovementCounter > VerticalMovement-
Threshold

pathCost = FindLowestCostPath(dtwMatrix)

// Calculate the average Pathcost per image

minPathcostPerFrame = pathCost / gesture.Frames.Count

// Maximum cost accepted per frame = 30

If (minPathcostPerFrame < 30 and pathCost $\neq -1$): return **g_{recogn}** // name to be displayed on screen

else

return none;

end

end

a search in the DTW matrix for the shortest path, in the virtual space of frames (observed versus reference gestures), as illustrated in Fig. 7, the neighboring values of static gestures are in general equal [Fig. 7(a)]. Thus, any direction in the DTW matrix could be chosen and the associated gesture could be falsely rejected based on the path taken. To avoid this problem, the search is biased (diagonal weight w_d is lower than vertical, w_v , and horizontal, w_h , weights) to favor the next image of both reference and last gesture performed, corresponding to a diagonal movement in the DTW matrix. In Fig. 7, values in circles inscribed in each cell represent the hand disparity for each pair of the observed and stored gestures before applying the DTW direction weight (w_h , w_v , and w_d). The product of these weights with the circled values gives the value used for minimal cost computation, indicated in color on the bottom right of each cell.

To further improve the performance, a maximum number of ten horizontal and vertical movements are allowed. This results in a faster elimination of divergent candidate gestures. In order to select the winning gesture among the gesture candidates, the one with the smallest disparity value (and that respects the threshold) is chosen.

The pseudocode summarizing the approach is shown in Algorithm 4.

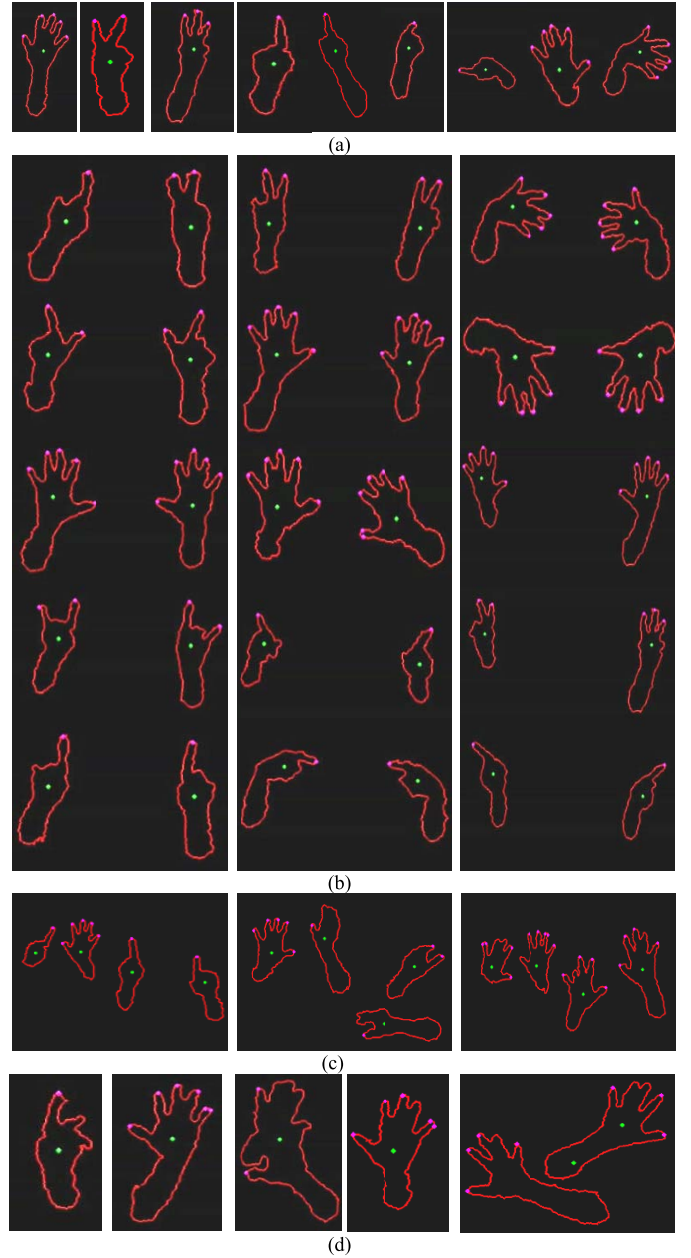


Fig. 8. Examples of contour, palm center, and fingertip detection for (a) one hand gestures, (b) two hand gestures, (c) multiple hand gestures, and (d) few problematic cases.

III. EXPERIMENTAL RESULTS

Several experiments were performed in order to test the proposed system for hand tracking and the localization of the palm center and of the fingertips. The approach is tested and evaluated over a series of 55 static and dynamic gestures for which the results are reported in the literature to enable performance comparisons.

A. Hand Detection and Tracking

A few samples for the detection of the contour, palm, and fingertips for one hand are shown in Fig. 8(a), for two hands in Fig. 8(b), and for multiple hands in Fig. 8(c). It can be observed that, in general, the proposed solution is able to

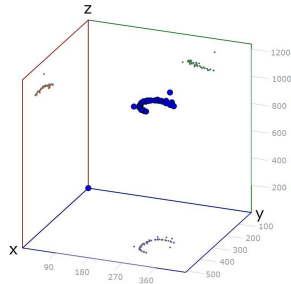


Fig. 9. Relative position of the palm center for dynamic circle gesture (in blue) and its projections over the x , y , and z axes. z represents the depth.

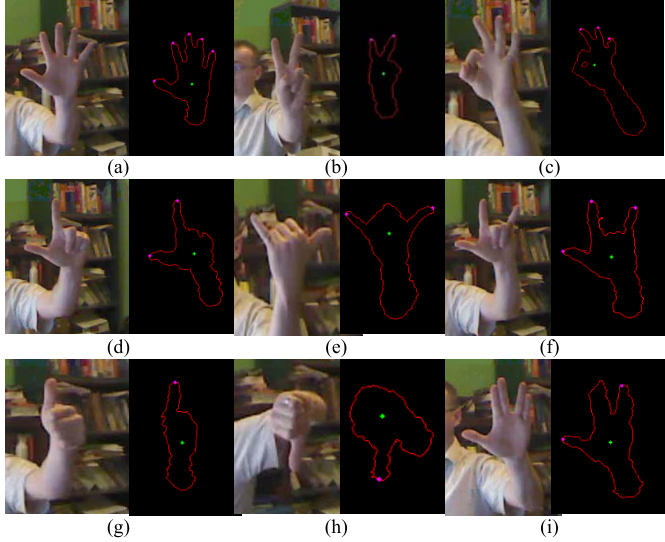


Fig. 10. Popular gestures. (a) Complete hand. (b) Victory. (c) OK. (d) Letter L. (e) Letter Y. (f) I love you. (g) Thumb up. (h) Thumb down. (i) Star Trek.

correctly locate the points of interest over the hand surface as well as its contour. A few problematic cases were also encountered with the proposed solution. Some examples are illustrated in Fig. 8(d). The palm center, fingers, and contour are sporadically lost between two consecutive images, and this behavior is related to the Kinect sensor. Fig. 9 illustrates this problem by showing the position of the palm center, in pixels, during the execution of a circle-like dynamic gesture. To deal with the issue of missing measurements along the z -axis (the blue points located at the origin in Fig. 9) and of other outliers, the proposed solution identifies and eliminates them from the data before proceeding with gesture recognition.

Sometimes double fingertips are identified for the same finger. This is mainly due to the use of the k -curvature algorithm, which, because of implementation specificities, tends to require larger distances between two fingers in order to make the distinction between them. This problem can be addressed, for example, by selecting the points that are the furthest from the palm center. Fingertips can be missing, in particular for gestures involving heavily bending of fingers or when occlusions occur due to the superposition of hands. The latter characterizes all optical motion capture sensors, but has less impact on the proposed application, since gestures do not involve occluding hands.

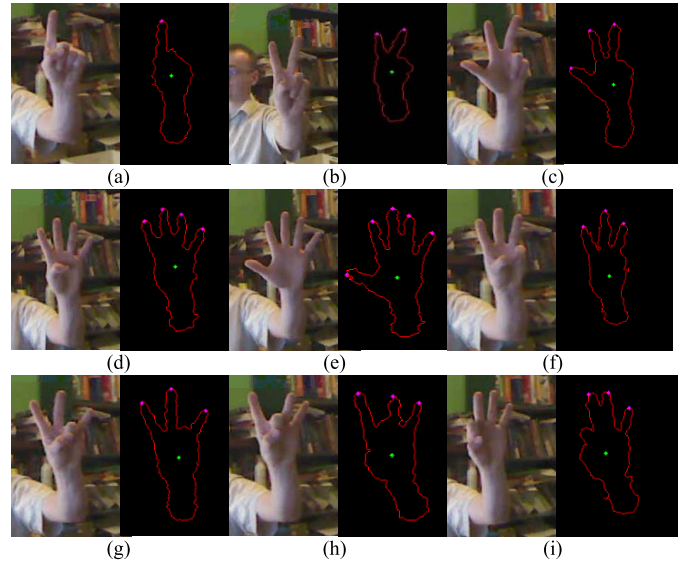


Fig. 11. Sign number gestures. (a) One. (b) Two. (c) Three. (d) Four. (e) Five. (f) Six. (g) Seven. (h) Eight. (i) Nine.

B. Static Gesture Recognition

In order to test the performance of the proposed solution, we have used a set of nine popular gestures, illustrated in Fig. 10, as proposed in [24], the set of sign digits from one to nine shown in Fig. 12 as in [7], [18], and [24], and the ASL alphabet [9], [21], [27]. Figs. 10 and 11 show, for the first two testing sets, the image of each gesture and the identified contour, palm center, and fingertips when only the relative position of the fingers with respect to the palm is used for recognition. The tests do not take place in controlled conditions. As one can observe in Figs. 10 and 11, the background is heavily cluttered. To perform the experimental testing, for each sign in each test set, reference gestures of a user are registered in double: one version roughly situated at 950 mm and the other one at roughly 1050 mm. The purpose of this double recording is to compensate for the scaling effect due to the use of different distances of the hand with respect to the camera. Reference gestures are separately recorded for each category of tests. The first part of testing is single blind. The test user is given a sequence of images corresponding to different gestures, and he/she is asked to reproduce them. Each test user is allowed to practice each gesture once before proceeding with the tests. He/she is then provided in real time with the recognition results.

During the second part of the test, the user is asked to reproduce gesture of a sequence of images, but he/she is not informed whether the gesture was recognized or not (double blind testing). For each type of test, single or double blind, each set of tests is repeated five times by each of the nine test users and the corresponding color video is recorded to calculate offline the time required for the recognition. A gesture is considered unrecognizable if the system is unable to identify it within a predefined interval after the beginning of its execution. If the decision fluctuates between two reference gestures, the observed gesture is considered recognized when the system

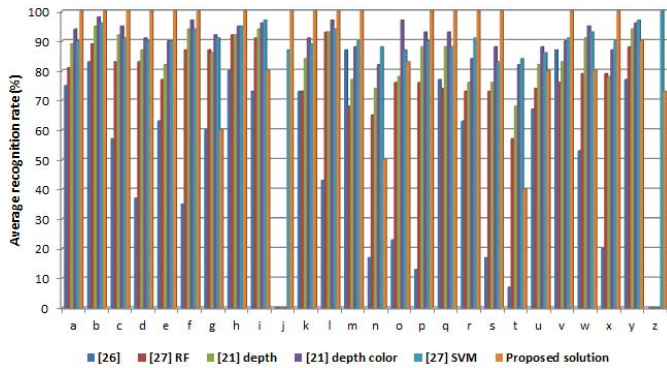


Fig. 12. Comparison for the gestures in the ASL alphabet.

TABLE II
AVERAGE RECOGNITION RATES AND TIME
ESTIMATES FOR SIGN NUMBERS

Gesture	Single blind		Double blind	
	Rate (%)	Time(s)	Rate (%)	Time(s)
One	100	1.63	100	1.48
Two	100	1.58	100	2.03
Three	80	5.30	90	3.36
Four	100	2.18	95	2.43
Five	80	4.93	80	4.88
Six	90	4.01	75	3.81
Seven	95	2.83	90	3.37
Eight	100	1.85	100	1.68
Nine	100	1.80	100	2.08
Average	93.9	2.9	92.2	2.8

TABLE III
AVERAGE RECOGNITION RATES AND TIME
ESTIMATES FOR POPULAR GESTURES

Gesture	Single blind		Double blind	
	Rate (%)	Time(s)	Rate (%)	Time(s)
Open hand	80	4.93	70	1.26
Victory/ letter V	95	1.49	100	1.63
OK/ letter F	40	2.69	40	3.81
Letter L	100	1.65	100	2.03
Letter Y	90	1.53	90	3.16
I love you	100	2.80	100	3.45
Thumbs up	100	2.03	100	2.13
Thumbs down	100	2.83	90	4.21
Star Trek	60	4.3	40	1.81
Average	85	2.7	81.1	2.6

decision stabilizes for one of the candidates. If the system does not stabilize within the interval, it is considered that the corresponding gesture was not identified. The results of this analysis, when using only fingertip information, are shown for the single- and double-blind tests in Tables II and III, respectively.

One can observe that the average performance during the double blind testing is slightly lower than for the one obtained for the single-blind tests. The slight decrease is justified by the lack of feedback to the user, which will not normally be the case of any natural gesture interface. The tests are nevertheless

TABLE IV
AVERAGE RECOGNITION RATES (%) FOR SIGN NUMBERS

Gesture	State-of-the-art solutions			Proposed solution***
	[24]	[7]*	[7]**	
One	91.75	92	96	84
Two	89.25	82	86	85
Three	90	91	94	75
Four	100	86	88	71
Five	100	85	92	82
Six	77	93	96	84
Seven	84.25	91	96	82
Eight	76.25	89	93	80
Nine	74	97	98	80
Average	86.9	89.6	93.2	80.3

*threshold decomposition solution, ** near-convex decomposition solution

performed to show that, even in such conditions, without any user adaptation, the performance of the proposed solution does not degrade significantly (overall less than 4% single-blind versus double-blind variation). In terms of recognition time, there is no significant difference between the two types of tests (roughly 0.1 s). It is important to state that the time values reported in Tables II and III include the visual presentation of the tracking of the contour, palm, and fingertips, not only the computation time required by the recognition module. Tables IV and V compare the single-blind tests with related solutions in the literature.

This comparison gives only a general idea of the performance, because overall, very few details are provided in the literature on the specific testing scenarios employed. Judging by the details provided, our approach seems closer in testing procedure to the one in [24], with three significant differences: 1) the distance between the camera and the hand is larger in our tests (950–1050 versus 500–800 mm); 2) in [24] the results are calculated for both the left and the right hand; and 3) tests in [24] seem to be related to a single blind style of testing. It can be observed in Table IV that for the sign number recognition, our method outperforms most of the existing solutions, including the one in [24]. For the popular gestures, the proposed solution is in line with the existing literature, as it can be observed in Table V.

The lowest performance was obtained for the more difficult to execute gestures OK and Star Trek, especially in the case when only the relative position of fingertips is used. For the Star Trek sign, the lower rate obtained by our solution might be explained by the fact that in the case of [24], only two of four people were able to execute the gesture and the results are reported for only these two, while in our case, the users were asked to imitate the gesture to their best possible and all the user results were included in the evaluation even if the users were not able to execute the gesture correctly. Overall, the performance improves when the contour is used as well in the decision making process, even for gestures including fingertips (e.g., the results obtained for the OK sign in the last two columns of Table V). This can be explained by the fact that, according to the execution of a gesture by different users, the fingertips can be more or less visible or even lost

TABLE V
RECOGNITION RATES (%) FOR POPULAR GESTURES

Gesture	State-of-the-art solutions					Proposed solution	
	[24]	[26]	[16]	[21]*	[27]*	**	***
Open hand	99	-	-	-	-	80	80
Victory (V)	89.2	87	83	90	91	95	100
OK (F)	84.8	35	94	97	94	40	100
Letter L	91	87	95	96	94	100	100
Letter Y	90.5	77	94	96	97	90	90
I love you	85.2	-	-	-	-	100	100
Thumbs up	89.5	-	-	-	-	100	100
Thumbs down	88.2	-	-	-	-	100	100
Star Trek	84	-	-	-	-	60	60
Average	89.1	71.5	91.5	94.8	94	85	92.2

- no data available, * best solution, ** fingertips, *** fingertips and contour

during tracking, therefore engaging the use of the contour information. The addition of the contour comes, however, at a slightly higher computation cost (Section III-E). Fig. 12 compares the performance of the static gesture recognition for the ASL alphabet executed with one hand and using the approach that includes the contour, with solutions proposed in the literature. The same testing procedure is used as in the case of popular gestures and sign numbers, but using only single blind conditions. The missing results for the j and z signs in Fig. 12 are due to the fact that these two letters are dynamic and several approaches work only on static data. It can be observed that the proposed solution (last column) is similar in performance to [27] for their SVM approach (penultimate column) with 89.85% average recognition rate over the alphabet letters for our solution and 90.85% in [27].

One can observe that the proposed solution has difficulties in differentiating between similar gestures such as m and n , and s and t . These could be addressed using the color camera of the Kinect to provide extra information about the hand or using a higher resolution in the processing of each frame in the contour algorithm. These both come with an additional computational burden. Alternatively, the z -axis information could be used, but this implies that the distance from the user to the Kinect has to be reduced in order to ensure a better contour identification. However, there is no guarantee that overall the results will improve due to the problem of missing measurements along the z -axis discussed in Section III-A.

Table VI compares the performance with the solution of Zhu and Yuan [17] for the set of two hand gestures reported in their work. The performance of the proposed solution is in this case better.

A final test for static gestures was performed for the context of natural control of a software application, as shown in Fig. 13. In particular, the thumb-down gesture was recorded as a reference gesture to close the interface of our system.

C. Dynamic Gesture Recognition

Additional tests were performed to recognize different dynamic gestures. A few test gestures are shown in Fig. 14, and the average recognition rates and a comparison of dynamic

TABLE VI
RECOGNITION RATES (%) FOR TWO HAND GESTURES

Gesture	[17]	Proposed solution
Both palms	99	100
Both fists	97	100
Left fist, right palm	98	100
Left palm, right fist	98	100
Left hand above right hand	99	100
Right hand above left hand	99	100
Average	98.3	100

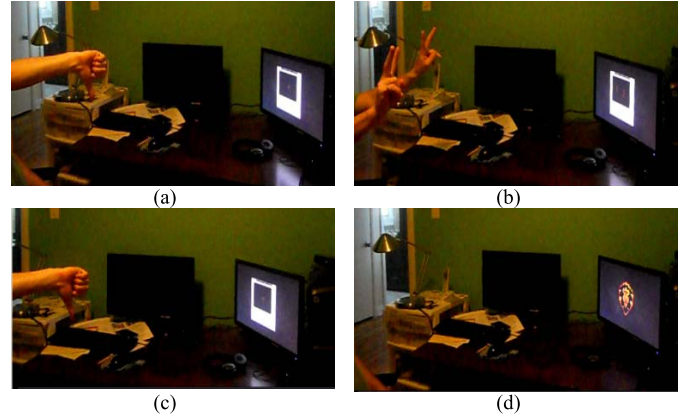


Fig. 13. Natural gesture software application control. (a) Training the system with the thumb-down gesture to close the application. (b) Series of other gestures is performed. (c) Thumb-down gesture is executed. (d) Application is closed 5 s after the gesture is recognized.

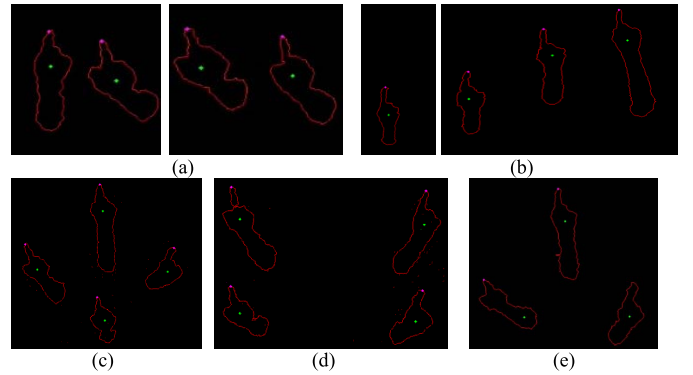


Fig. 14. Dynamic gestures. (a) Click. (b) Going up. (c) Circle. (d) Rectangle. (e) Triangle.

and static gestures are shown in Table VII. The results are presented for the proposed approach using single blind testing following the same procedure used for popular gestures and sign numbers.

One can observe that overall the system obtains good recognition rates for dynamic gestures as well. It is important to state that unlike the system proposed in [24], the one proposed in this paper (the case of relative position of fingers is reported) is performing the recognition in real time.

D. Distinction Between Static and Dynamic Gestures

Because the same solution is used to deal with both dynamic and static gestures, additional testing was performed to test

TABLE VII
RECOGNITION RATES (%) FOR ONE HAND DYNAMIC GESTURES

Gesture	State-of-the-art solutions				Proposed solution
	[32]	[34]	[29]	[17]	[3]
Circle	100	-	97.8	88	-
Bye	94.4	90	-	-	96.3
Rectangle	94	-	-	-	-
Triangle	99.4	80	-	-	-
Going up	-	-	100	94	89.2
Going down	-	-	100	93	91.9
Going left	-	-	97.8	95	96
Going right	-	-	100	95	95.5
Average	96.95	85	99.12	93	93.72

- no data available

TABLE VIII
AVERAGE RECOGNITION RATES (%) FOR DTW FOR
VALIDATION AND PROPOSED SOLUTION

Gesture description	[14]	Proposed solution
Distinction between click and immobile finger	80	100
Distinction between two hands with one immobile finger each and two simultaneous clicks executed with the finger of each hand	100	80
Distinction between immobile and moving hand	20	80
Average	66.6	86.6

TABLE IX
AVERAGE TIME ESTIMATES PER FRAME (ms)

Solution	Computer configuration	No. of gestures	Time per frame
[9]	1.83Ghz processor	26	62.2ms
[29]	3Ghz processor, 1GB RAM	7	60ms
[19]	2.66Ghz processor, 3.3GB RAM	10	70ms*
[35]	3Ghz processor, 3GB RAM	10	87ms
Proposed solution	2.3Ghz processor, 3.3GB RAM	55	33ms**/ 40ms***

*threshold decomposition, ** fingertips, *** fingertips and contour

the capability of the system to distinguish between the two as well as to compare with the case when the DTW is used only for validation, as in [14]. As stated in Section II-D, the DTW has in this paper a double role: to identify candidate gestures and to validate the similarity. Table VIII compares the proposed solution with the case when the DTW is employed only for validation. One can observe that, in spite of the fact that for our solution, some difficulties are encountered with fast movements (e.g., fast consecutive clicks in the first line of Table VIII) and sometimes also in distinguishing between similar static and dynamic gestures, the results are overall better than when using DTW for validation only. The latter is affected by gestures that have similar initial and final frames.

E. Average Execution Time

Table IX shows the performance, in terms of the average execution time per frame, achieved by the proposed solution

with respect to other state-of-the-art approaches. In spite of the fact that a direct comparison on the same computer is not possible, it can be observed that the proposed solution is faster when compared with most of the approaches from the literature on an almost similar computing platform.

IV. CONCLUSION

This paper presented the development of a novel system for gesture recognition using depth images collected by a Kinect sensor. Starting from the closest pixel in the scene, the initial pixel of the hand contour is found using an improved block search scheme and a directional search is performed for the identification of the entire hand contour. The k -curvature algorithm is then employed to locate the fingertips over the contour. Based on the hand position, the DTW algorithm is used to select a candidate gesture as well as to recognize gestures by comparing them with a series of prerecorded reference gestures. The system achieves an average performance of 92.4% for one or multiple hand static and dynamic gestures, with which it is able to deal simultaneously. As only the depth sensor of the Kinect is employed, the background, the lighting conditions, the clothing, and the skin color have little impact on the results obtained by the proposed solution. Nevertheless, during testing, some problems were encountered when one of the users wore a bracelet.

As future improvements, an additional module will be added to allow for a more accurate adaptation of the size of a user hand instead of the double training at different depths that is currently implemented in the system. This would improve the performance at the level of recognition stage. In the same direction, a different version of DTW [40] will be evaluated for further speeding up the gesture recognition.

The proposed solution has multiple applications within the realm of natural user interfaces, from which two were presented: one for the real-time interpretation of sign digits and popular gestures and the other one in the context of natural control of a software application.

REFERENCES

- [1] A. R. Várkonyi-Kóczy and B. Türos, "Human-computer interaction for smart environment applications using fuzzy hand posture and gesture models," *IEEE Trans. Instrum. Meas.*, vol. 60, no. 5, pp. 1505–1514, May 2011.
- [2] Q. Chen, N. D. Georganas, and E. M. Petriu, "Hand gesture recognition using Haar-like features and a stochastic context-free grammar," *IEEE Trans. Instrum. Meas.*, vol. 57, no. 8, pp. 1562–1571, Aug. 2008.
- [3] C.-C. Hsieh and D.-H. Liou, "Novel Haar features for real-time hand gesture recognition using SVM," *J. Real-Time Image Process.*, vol. 10, no. 2, pp. 357–370, Jun. 2015.
- [4] S. Mitra and T. Acharya, "Gesture recognition: A survey," *IEEE Trans. Syst., Man, Cybern. C, Appl. Rev.*, vol. 37, no. 3, pp. 311–324, May 2007.
- [5] J. Suarez and R. R. Murphy, "Hand gesture recognition with depth images: A review," in *Proc. IEEE Int. Symp. Robot Human Interact. Commun.*, Paris, France, Sep. 2012, pp. 411–417.
- [6] F. T. Cerezo, *3D Hand and Finger Recognition Using Kinect*. Granada, Spain: Univ. Granada, 2013.
- [7] Z. Ren, J. Yuan, and Z. Zhang, "Robust hand gesture recognition based on finger-earth mover's distance with a commodity depth camera," in *Proc. ACM Int. Conf. Multimedia*, 2011, pp. 1093–1096.
- [8] R. P. Mihail, N. Jacobs, and J. Goldsmith, "Static hand gesture recognition with 2 Kinect sensors," in *Proc. Int. Conf. Image Process., Comput. Vis., Pattern Recognit.*, vol. 2, 2012, pp. 911–917.

- [9] D. Uebbersax, J. Gall, M. Van den Bergh, and L. Van Gool, "Real-time sign language letter and word recognition from depth data," in *Proc. IEEE Int. Conf. Comput. Vis.*, Barcelona, Spain, Nov. 2011, pp. 383–390.
- [10] K. Kollorz, J. Penne, J. Hornegger, and A. Barke, "Gesture recognition with a time-of-flight camera," *Int. J. Intell. Syst. Technol. Appl.*, vol. 5, nos. 3–4, pp. 334–343, Nov. 2008.
- [11] K. Rimkus, A. Bukis, A. Lipnickas, and S. Sinkevicius, "3D human hand motion recognition system," in *Proc. IEEE Int. Conf. Human Syst. Interact.*, Sopot, Poland, Jun. 2013, pp. 180–183.
- [12] M. Correa, J. Ruiz-del-Solar, R. Verschae, J. Lee-Ferng, and N. Castillo, "Real-time hand gesture recognition for human robot interaction," in *RoboCup* (Lecture Notes in Computer Science), vol. 5949, J. Baltes, M. G. Lagoudakis, T. Naruse, and S. S. Ghidary, Eds. Berlin, Germany: Springer-Verlag, 2010, pp. 46–57.
- [13] M. Van den Bergh and L. Van Gool, "Combining RGB and ToF cameras for real-time 3D hand gesture interaction," in *Proc. IEEE Workshop Appl. Comput. Vis.*, Kona, HI, USA, Jan. 2011, pp. 66–72.
- [14] D. J. Ryan, "Finger and gesture recognition with Microsoft Kinect," M.S. thesis, Dept. Elect. Comput. Eng., Univ. Stavanger, Stavanger, Norway, 2012.
- [15] M. Tang, "Recognizing hand gestures with Microsoft's Kinect," *Stanfordedu*, vol. 14, no. 4, pp. 303–313, 2011.
- [16] I. Oikonomidis, N. Kyriazis, and A. A. Argyros, "Efficient model-based 3D tracking of hand articulations using Kinect," in *Proc. Brit. Mach. Vis. Conf.*, 2011, pp. 101.1–101.11.
- [17] Y. Zhu and B. Yuan, "Real-time hand gesture recognition with Kinect for playing racing video games," in *Proc. Int. Joint Conf. Neural Netw.*, Beijing, China, Jul. 2014, pp. 3240–3246.
- [18] Z. Ren, J. Meng, and J. Yuan, "Depth camera based hand gesture recognition and its applications in human-computer-interaction," in *Proc. IEEE Conf. Inf., Commun. Signal Process.*, Dec. 2011, pp. 1–5.
- [19] Z. Ren, J. Yuan, J. Meng, and Z. Zhang, "Robust part-based hand gesture recognition using Kinect sensor," *IEEE Trans. Multimedia*, vol. 15, no. 5, pp. 1110–1120, Aug. 2013.
- [20] M. Van den Bergh *et al.*, "Real-time 3D hand gesture interaction with a robot for understanding directions from humans," in *Proc. IEEE Int. Symp. Robot Human Interact. Commun.*, Atlanta, GA, USA, Jul./Aug. 2011, pp. 357–362.
- [21] K. Otiniano-Rodríguez and G. Cámara-Chávez, "Finger spelling recognition from RGB-D information using kernel descriptor," in *Proc. IEEE Conf. Graph., Patterns Images, Arequipa*, Peru, Aug. 2013, pp. 1–7.
- [22] M. Schroder, C. Elbrechter, J. Maycock, R. Haschke, M. Botsch, and H. Ritter, "Real-time hand tracking with a color glove for the actuation of anthropomorphic robot hands," in *Proc. IEEE-RAS Int. Conf. Humanoid Robots*, Osaka, Japan, Nov./Dec. 2012, pp. 262–269.
- [23] H. Takimoto, S. Yoshimori, Y. Mitsukura, and M. Fukumi, "Classification of hand postures based on 3D vision model for human-robot interaction," in *Proc. IEEE Int. Symp. Robot Human Interact. Commun.*, Viareggio, Italy, Sep. 2010, pp. 292–297.
- [24] Y. Li, "Multi-scenario gesture recognition using Kinect," in *Proc. IEEE Int. Conf. Comput. Games*, Jul./Aug. 2012, pp. 126–130.
- [25] X. Liu and K. Fujimura, "Hand gesture recognition using depth data," in *Proc. IEEE Conf. Autom. Face Gesture Recognit.*, May 2004, pp. 529–534.
- [26] N. Pugeault and R. Bowden, "Spelling it out: Real-time ASL finger-spelling recognition," in *Proc. IEEE Int. Conf. Comput. Vis. Workshops (ICCV Workshops)*, Barcelona, Spain, Nov. 2011, pp. 1114–1119.
- [27] F. Pedersoli, S. Benini, N. Adami, and R. Leonardi, "XKin: An open source framework for hand pose and gesture recognition using Kinect," *Vis. Comput.*, vol. 30, no. 10, pp. 1107–1122, Oct. 2014.
- [28] M. R. Abid, P. E. Meszaros, R. F. D. Silva, and E. M. Petriu, "Dynamic hand gesture recognition for human-robot and inter-robot communication," in *Proc. IEEE Conf. Comput. Intell. Virtual Environ. Meas. Syst. Appl.*, Ottawa, ON, Canada, May 2014, pp. 12–17.
- [29] X. Wu, C. Yang, Y. Wang, H. Li, and S. Xu, "An intelligent interactive system based on hand gesture recognition algorithm and Kinect," in *Proc. IEEE Int. Symp. Comput. Intell. Design*, Hangzhou, China, Oct. 2012, pp. 294–298.
- [30] N. H. Dardas and N. D. Georganas, "Real-time hand gesture detection and recognition using bag-of-features and support vector machine techniques," *IEEE Trans. Instrum. Meas.*, vol. 60, no. 11, pp. 3592–3607, Nov. 2011.
- [31] M. Vanco, I. Minarik, and G. Rozinaj, "Evaluation of static hand gesture algorithms," in *Proc. Int. Conf. Syst., Signals, Image Process.*, Dubrovnik, Croatia, May 2014, pp. 83–86.
- [32] M. R. Abid, E. M. Petriu, and E. Amjadian, "Dynamic sign language recognition for smart home interactive application using stochastic linear formal grammar," *IEEE Trans. Instrum. Meas.*, vol. 64, no. 3, pp. 596–605, Mar. 2015.
- [33] J. M. Palacios, C. Sagüés, E. Montijano, and S. Llorente, "Human-computer interaction based on hand gestures using RGB-D sensors," *Sensors*, vol. 13, no. 9, pp. 11842–11860, 2013.
- [34] Z. Li and R. Jarvis, "Real time hand gesture recognition using a range camera," in *Proc. Australasian Conf. Robot. Autom.*, Sydney, NSW, Australia, 2009, pp. 529–534.
- [35] Y. Yao and Y. Fu, "Contour model-based hand-gesture recognition using the Kinect sensor," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 24, no. 11, pp. 1935–1944, Nov. 2014.
- [36] F. Kobayashi and F. Kojima, "Handover motion based on human hand posture with hand/arm robot," in *Proc. IEEE Int. Symp. Micro-NanoMechatronics Human Sci.*, Nagoya, Japan, Nov. 2013, pp. 1–6.
- [37] P. Payeur *et al.*, "Human gesture quantification: An evaluation tool for somatic training and piano performance," in *Proc. IEEE Symp. Haptic Audio-Vis. Environ. Games*, Dallas, TX, USA, Oct. 2014, pp. 100–105.
- [38] E. Keogh and C. A. Ratanamahatana, "Exact indexing of dynamic time warping," *Knowl. Inf. Syst.*, vol. 7, no. 3, pp. 358–386, 2005.
- [39] M. Müller, "Dynamic time warping," in *Information Retrieval for Music and Motion*. New York, NY, USA: Springer-Verlag, 2007, pp. 69–82.
- [40] G. Al-Naymat, S. Chawla, and J. Taheri, "SparsedTW: A novel approach to speed up dynamic time warping," in *Proc. Australasian Data Mining Conf.*, vol. 101, 2012, pp. 117–127.



Guillaume Plouffe received the B.Sc. degree from the Université du Québec en Outaouais, Gatineau, QC, Canada. He is currently pursuing the master's degree with the University of Ottawa, Ottawa, ON, Canada.

His current research interests include human-machine interaction and gesture recognition.



Ana-Maria Cretu (S'04–M'10) received the M.A.Sc. and Ph.D. degrees from the School of Electrical Engineering and Computer Science, University of Ottawa, Ottawa, ON, Canada.

She is currently an Associate Professor with the Department of Computer Science and Engineering, Université du Québec en Outaouais, Gatineau, QC, Canada. She has authored over 60 technical papers. Her current research interests include computational intelligence, soft computing, biologically inspired computational models, tactile and vision sensing,

natural human-computer interaction, and 3-D object sensing, modeling, and manipulation.

Dr. Cretu is a member of the Editorial Board of the *Journal of Soft Computing* (Springer). She is a member of the IEEE Instrumentation and Measurement Society, the IEEE Computational Intelligence Society, and the IEEE Systems, Man and Cybernetics Society. She serves as a Technical Committee Member for several international conferences, and a Conference Organizer and Reviewer for several journals and transactions.